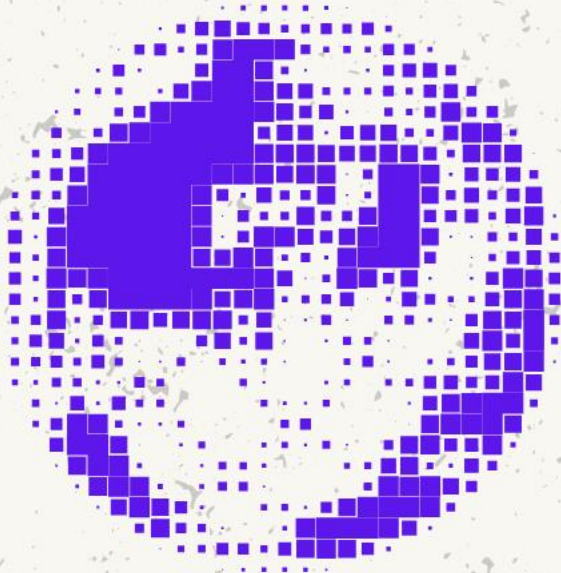




CVision

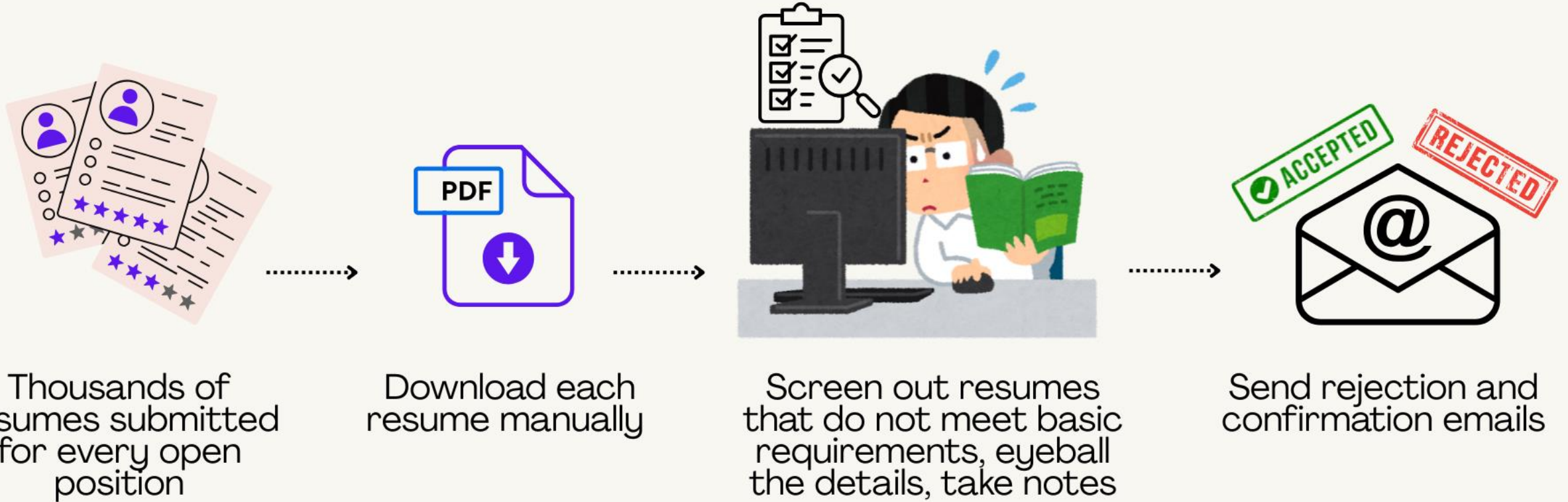
An Agentic Resume Screening Solution



Team 03

Tasnim Talha | Roll: 05
Talha Ibna Banna | Roll: 13
Nabiha Nuha | Roll: 56

Situation Analysis



***HR LOSING CONTROL
OVER THEIR TIME***

The Problem

The solutions in place are, frankly, inadequate.

Keyword matching: Traditional ATS software scans for exact keyword hits; great candidates can get silently filtered out.

No semantic ranking: Most ATS tools give a binary pass/fail against hard filters, not a nuanced score of how well a resume matches the job description.

No interactivity: A recruiter can't ask an ATS "who here has real NLP experience?" They have to compare resumes manually.

Manual, delayed communication: Sending acceptance/rejection emails is usually a separate, manual step recruiters put off, hurting candidate experience.



***BOTTLENECK IN
RECRUITMENT***

The Opportunity



A system that can pull in job applications, read the resumes, rank them against a job description, let a recruiter chat with an AI about the results, and auto-send decision emails.

The Solution



MEET *CVISION*

Presentation

app.py: Streamlit
Dashboard

Processing

ingestion · parser ·
embedding ·
chatbot ·
email_dispatch

Automation

scheduler_task.py +
pipeline_core.py

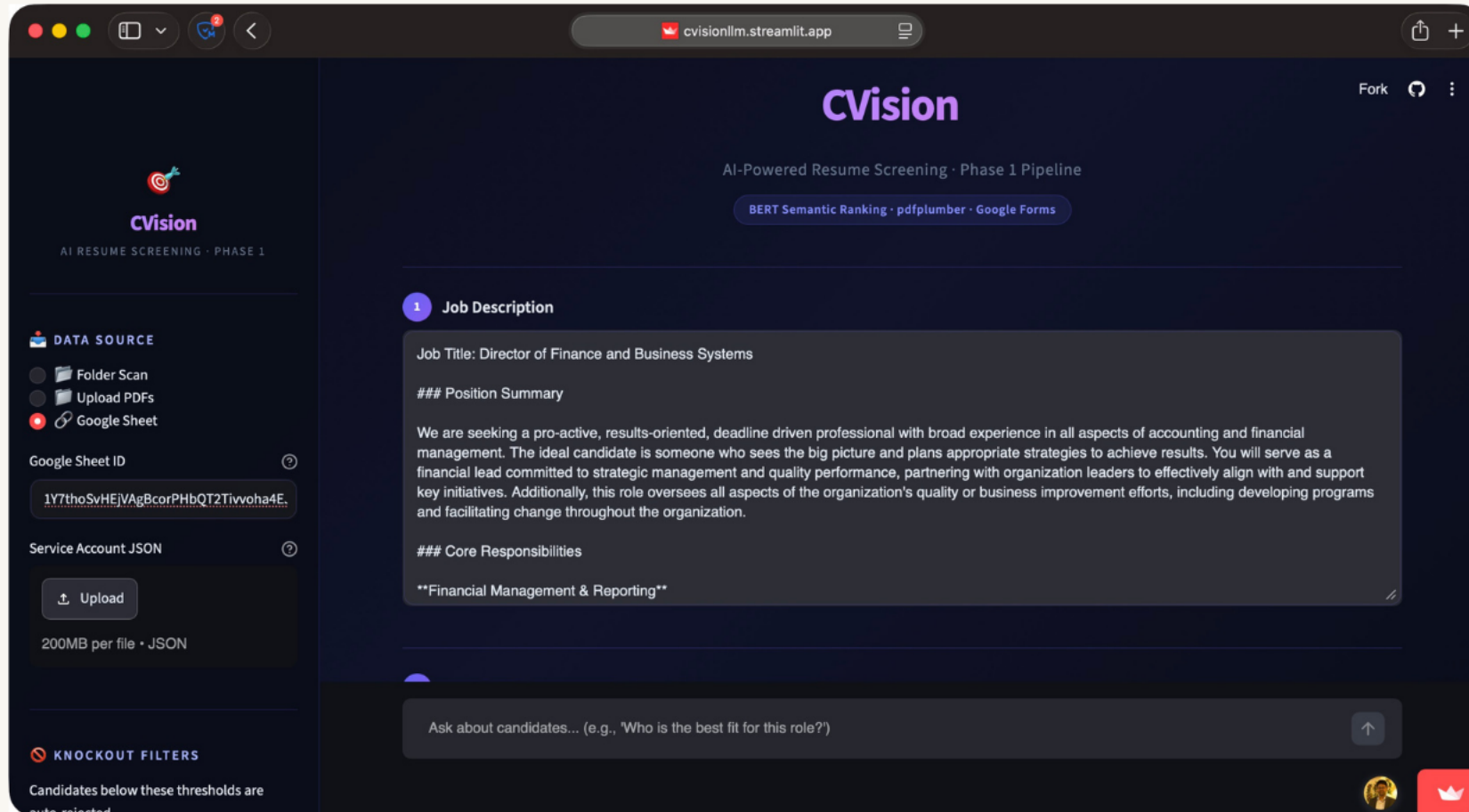
Semantic Screening: Automatically parses, filters, and ranks multi-column PDFs against the JD.

Agentic AI Assistant: Interact with a Gemini-powered agent to compare candidates, search skills, and uncover insights.

Automated Dispatch: Triggers personalized acceptance or rejection emails instantly or at a scheduled time.

The Frontend

The screen the recruiter sees

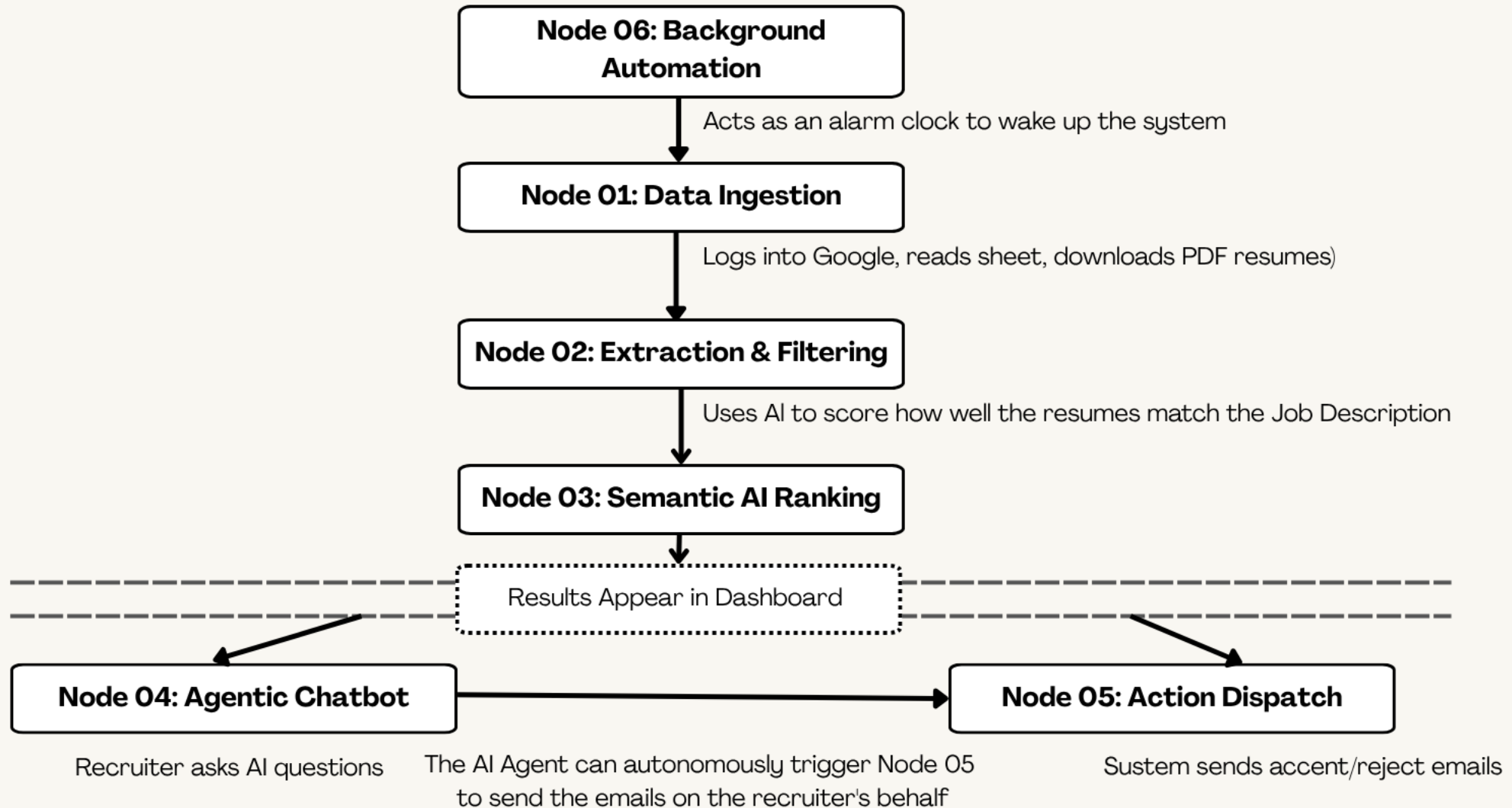


app.py

streamlit

System Architecture

The automated, end-to-end pipeline



Live Demo

<https://cvisionllm.streamlit.app/>



Node 01 — Data Ingestion

Entering the Pipeline

`modules/ingestion.py`

`gsread`

`google-auth`

`google-api-python-client`

The Challenge

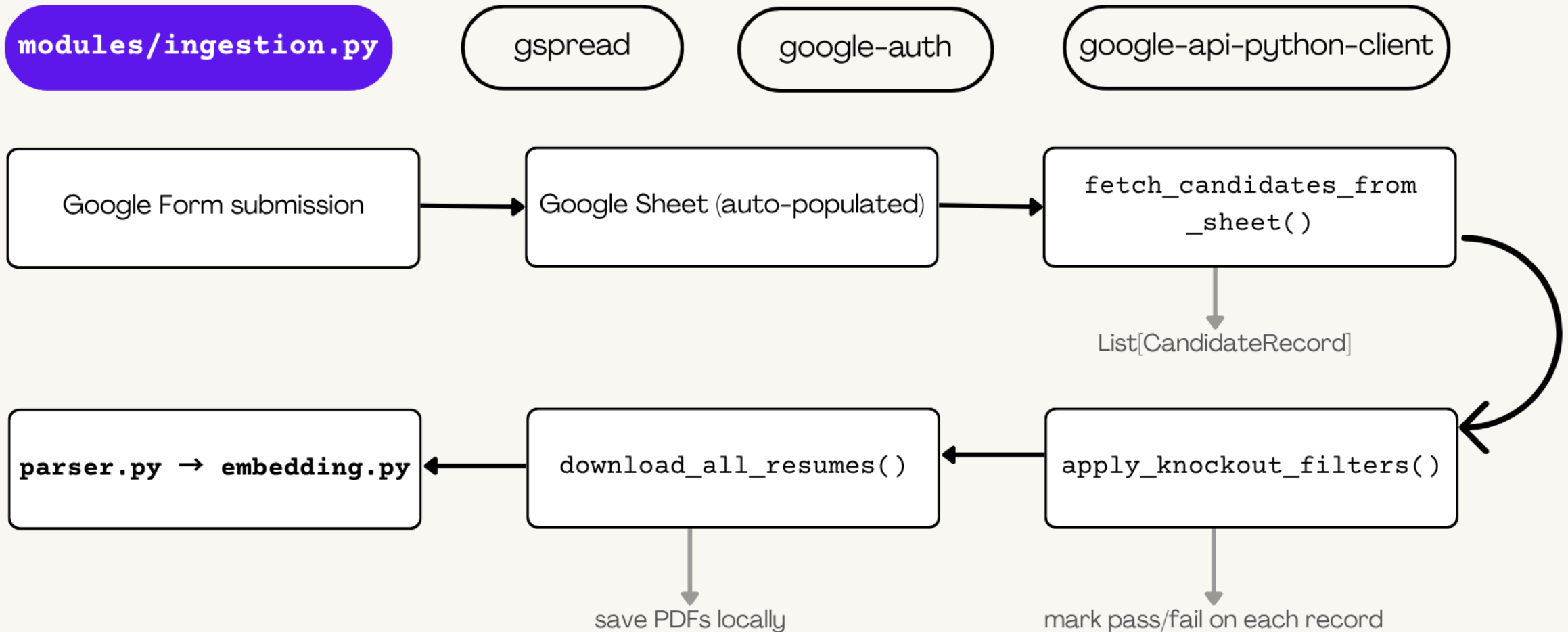
- Applications submitted via Google Forms
- Data scattered across Google Sheets and Drive
- Manual tracking is time-consuming

The Process

- System acts as an autonomous virtual recruiter
- Securely logs into Google Workspace
- Scans spreadsheet for new applicants
- Automatically downloads PDF resumes for processing

Node 01 — Data Ingestion

Collecting applications and running the first pass/fail check



Node 02 — Extraction & Filtering

Scanning & making sense of PDFs

`modules/parser.py`

pdfplumber

pypdf

re

The Challenge

- PDFs are visual and hard-to-read for machines
- Unstructured data layout

The Process

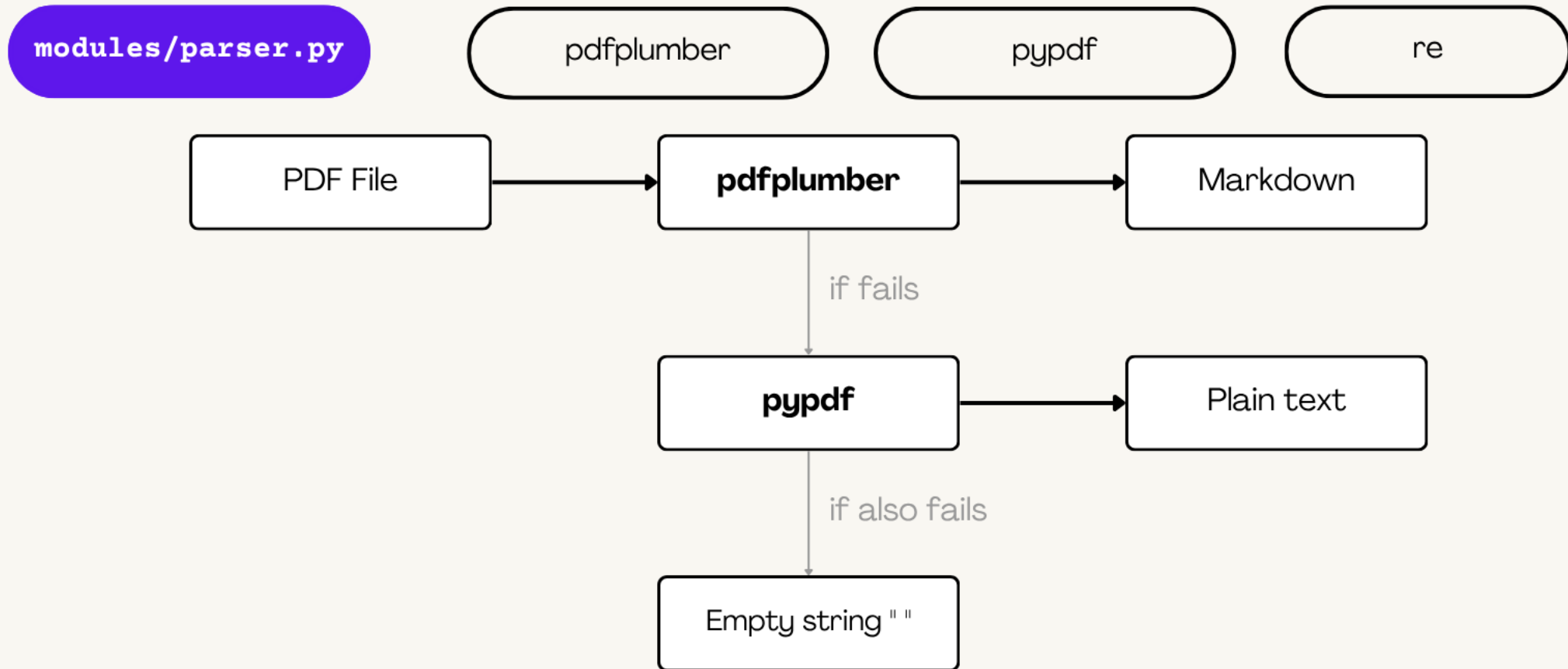
- Scans resumes to extract raw text
- Preserves formatting (bullet points, headings)

Knockout Phase

- Instantly checks critical metrics (CGPA, Experience)
- Auto-rejects unqualified applicants to save time

Node 02 — Extraction & Filtering

Turning a PDF into something a model can actually read



Node 03 — Semantic AI Ranking

The Semantic Brain

`modules/embedding.py`

all-MiniLM-L6-v2

fastembed

The Challenge

- Traditional systems rely on easily-tricked **Keyword Matching**
- Candidates can hide keywords in white text

The Process

- Uses neural networks to **understand** resume context
- Compares resumes directly against the Job Description
- Generates a 0–100% **Fit Score** based on actual meaning
- Immune to keyword stuffing

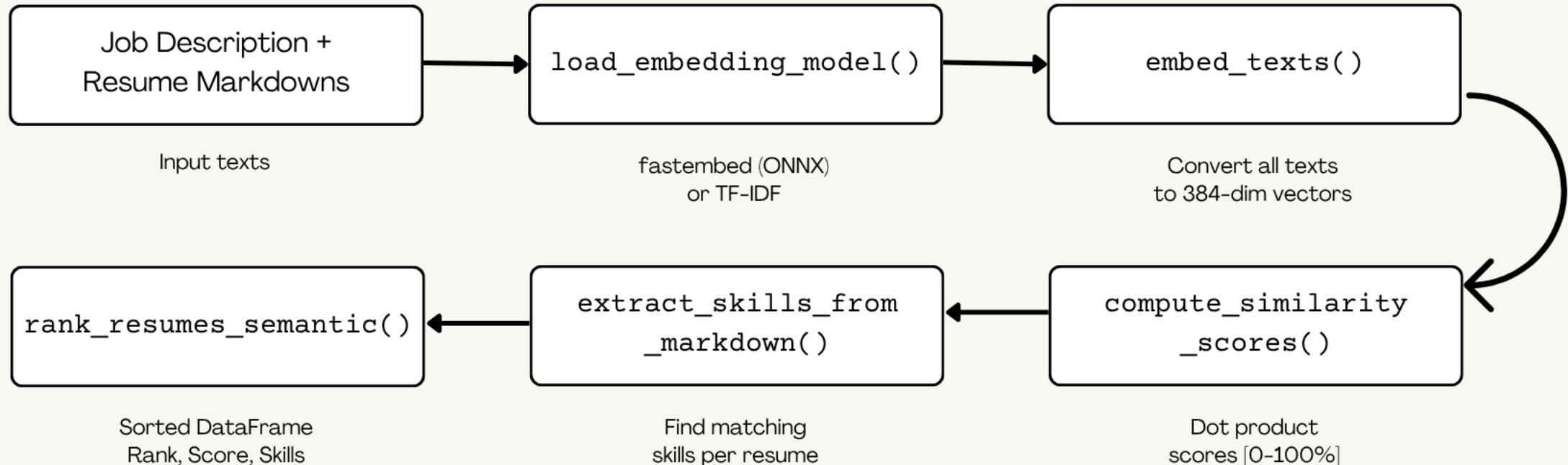
Node 03 — Semantic AI Ranking

Scoring every resume against the job, 0 to 100%

`modules/embedding.py`

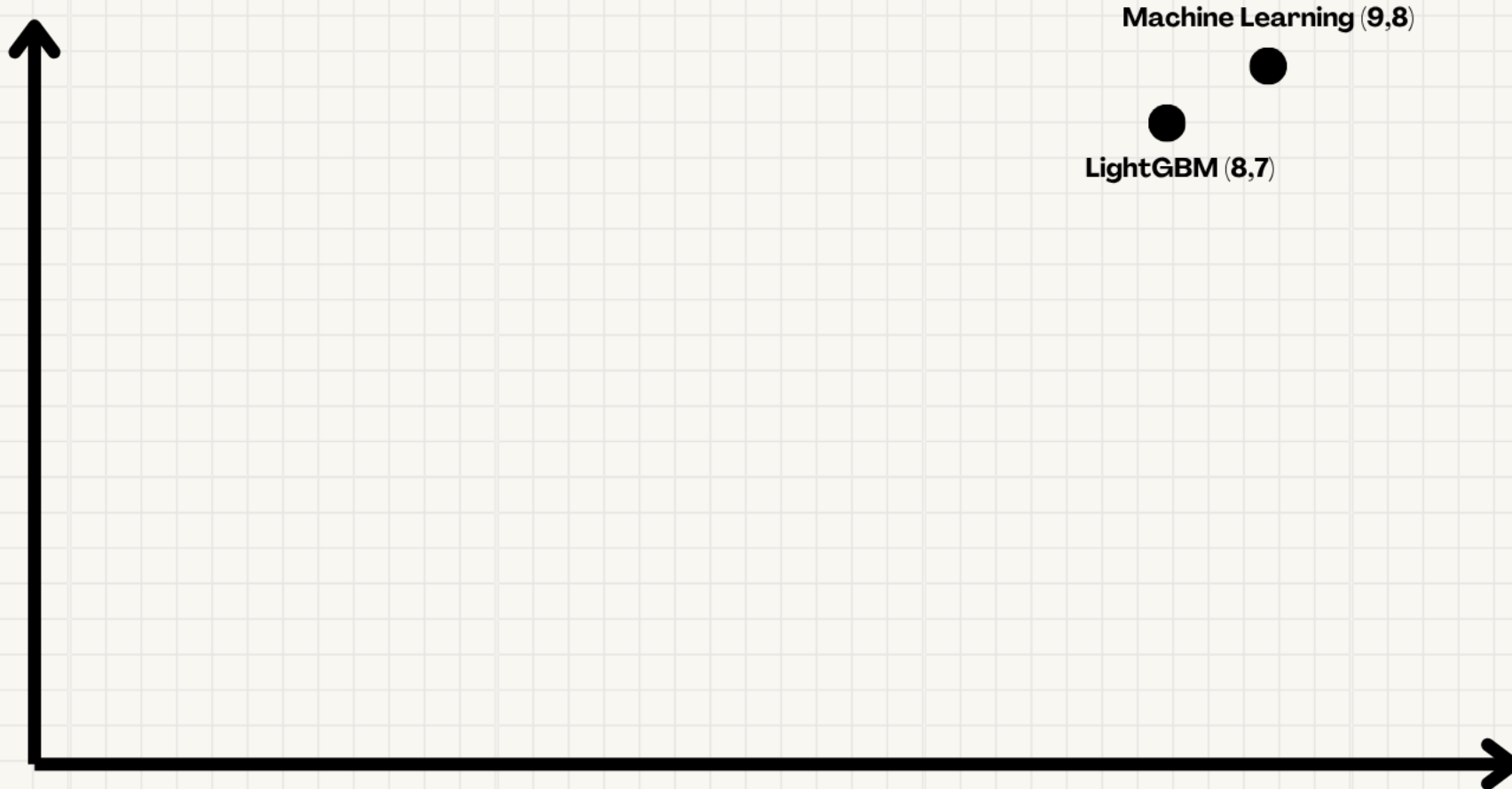
all-MiniLM-L6-v2

fastembed



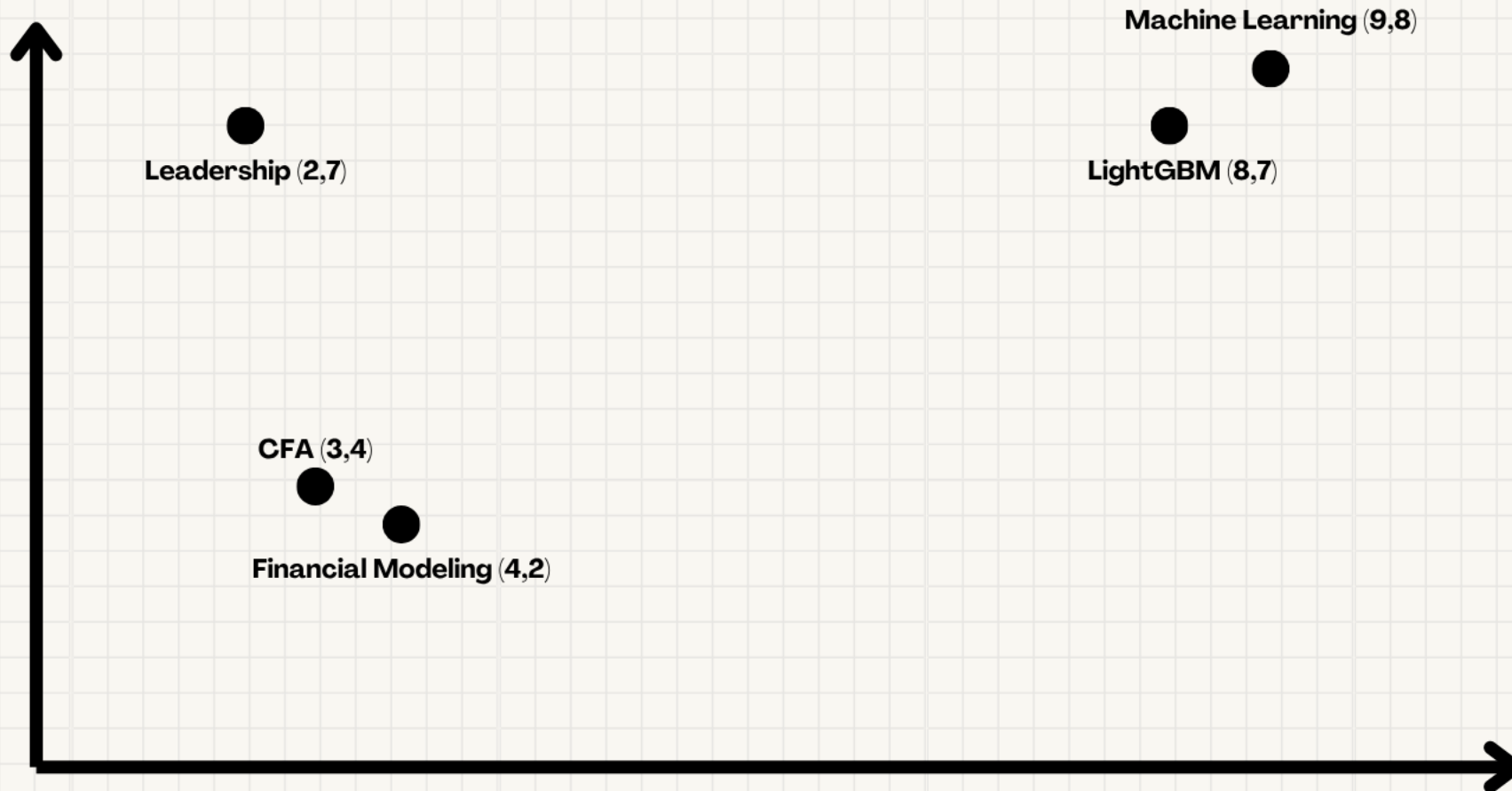
Node 03 — Semantic AI Ranking

How it works



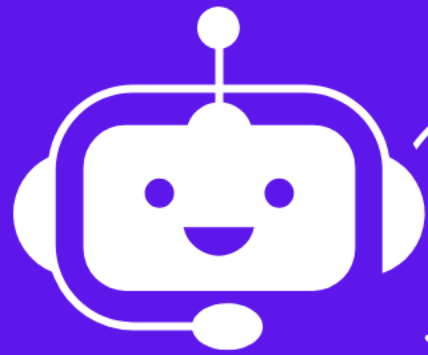
Node 03 — Semantic AI Ranking

How it works



The Agent (Node 4)

Our agent is composed of the **3** parts:



LLM: Google Gemini

Memory: Stored in Streamlit

Tools: SDK handles if/when functions

The system follows a **Hub & Spoke** model



Node 04 — The Agentic Chatbot

The AI Co-Pilot

`modules/chatbot.py`

google-genai

SDK

The Concept

- Acts as a true conversational partner, not just a text generator
- Operates autonomously as an AI **Agent**

Key Components

- **LLM (Brain):** Uses *lean-prompts* (summary only) and fetches full resumes on demand
- **Context Memory:** Remembers conversation history for natural follow-ups
- **Tools (Hands):** Can independently trigger functions to gather data, analyze gaps, or execute real-world actions

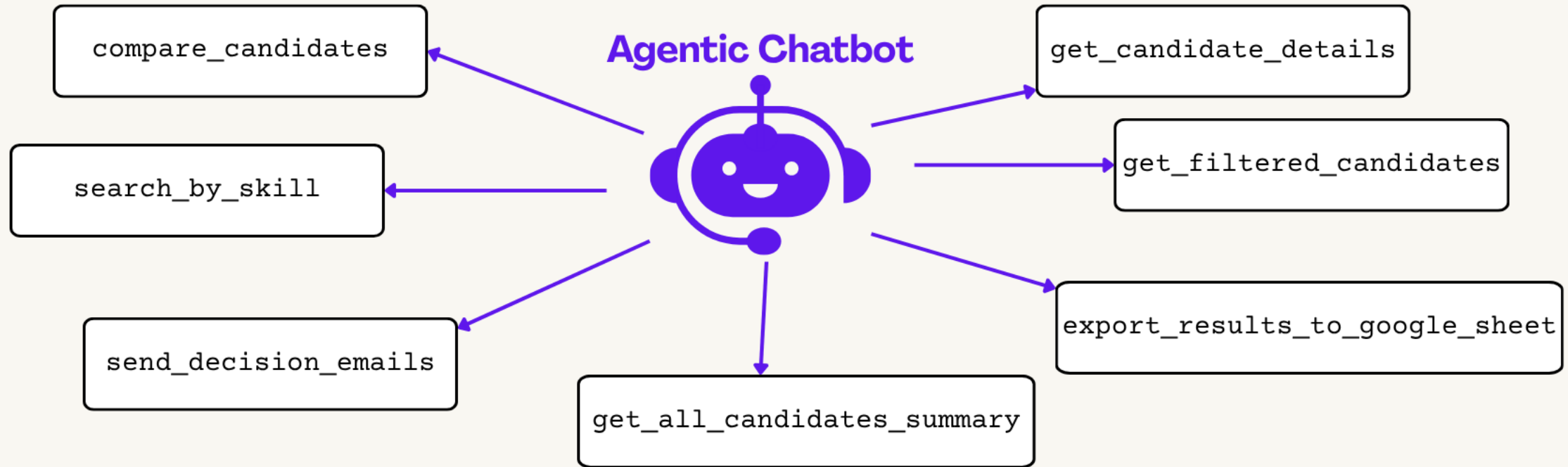
Node 04 — The Agentic Chatbot

How the AI Uses Tools

`modules/chatbot.py`

google-genai

SDK



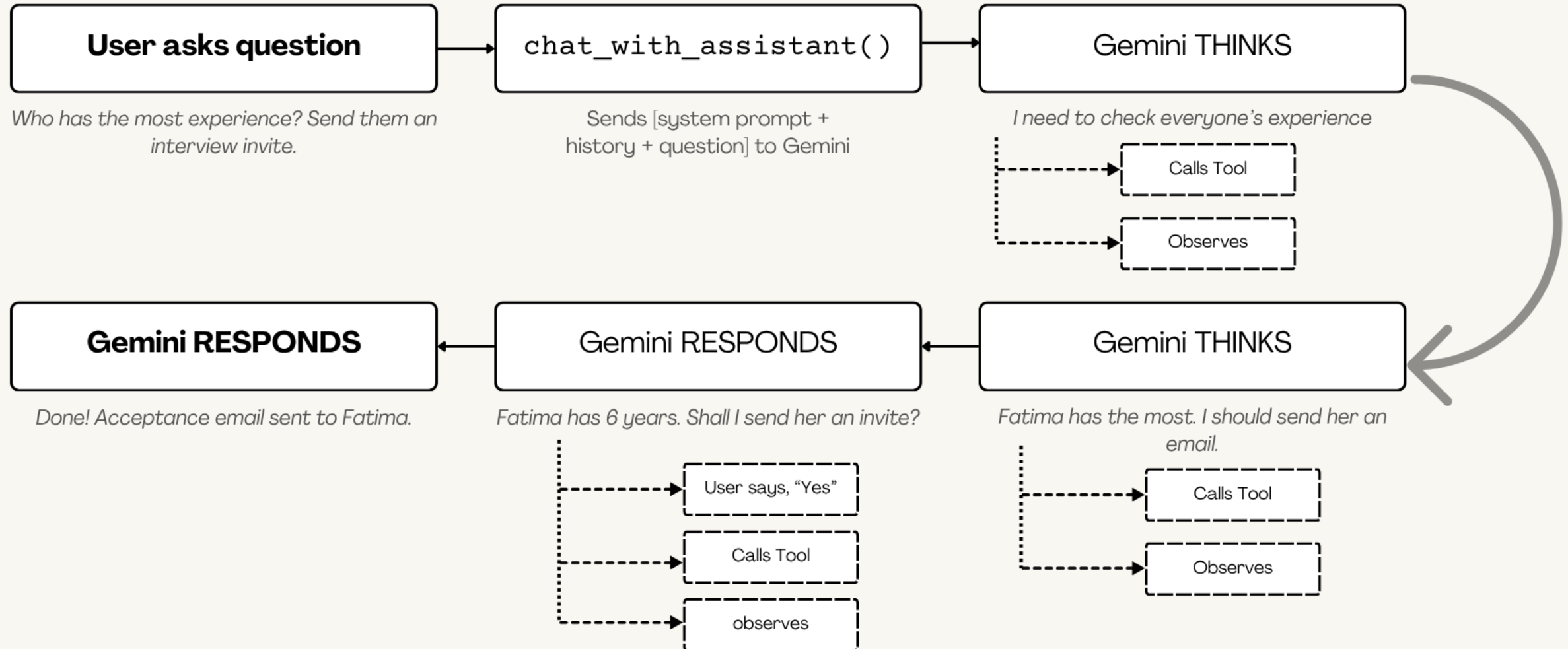
Node 04 — The Agentic Chatbot

Assistant that can look things up, and act with permission

`modules/chatbot.py`

google-genai

SDK



Node 05 — Action Dispatch

Executing Decisions

`modules/email_dispatch.py`

smtplib

gsread

The Challenge

- Need to communicate decisions to candidates
- Must maintain an updated system of record

The Process

- Dispatches customized Acceptance/Rejection emails
- Saves final ranking database and chat log back to the cloud

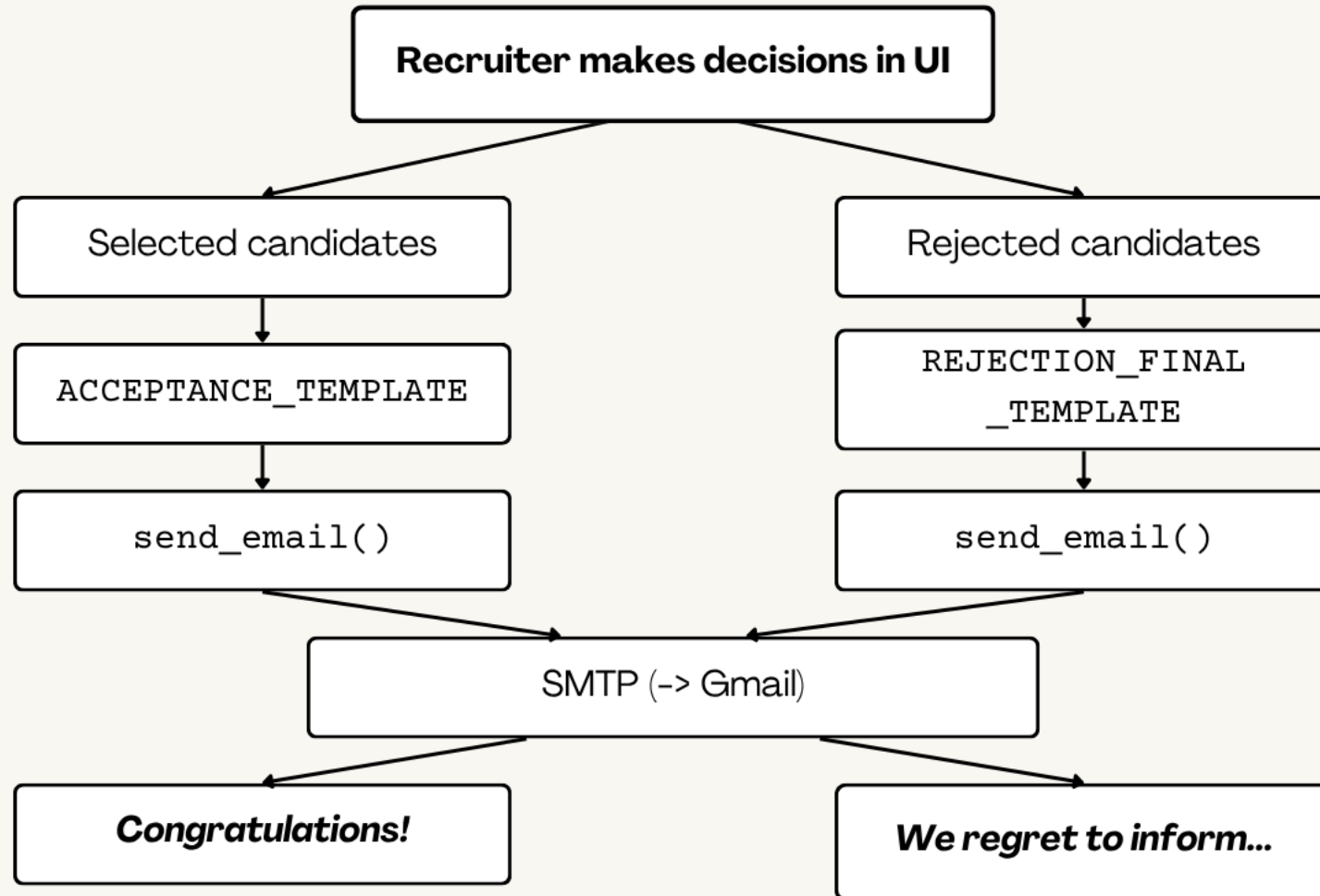
Agentic: AI autonomously triggers emails/exports via natural language

&

Manual: Dedicated UI buttons for recruiter review and reliable fallback

Node 05 — Action Dispatch

Sending the acceptance and rejection letters automatically



`modules/email_dispatch.py`

gsread

smtplib

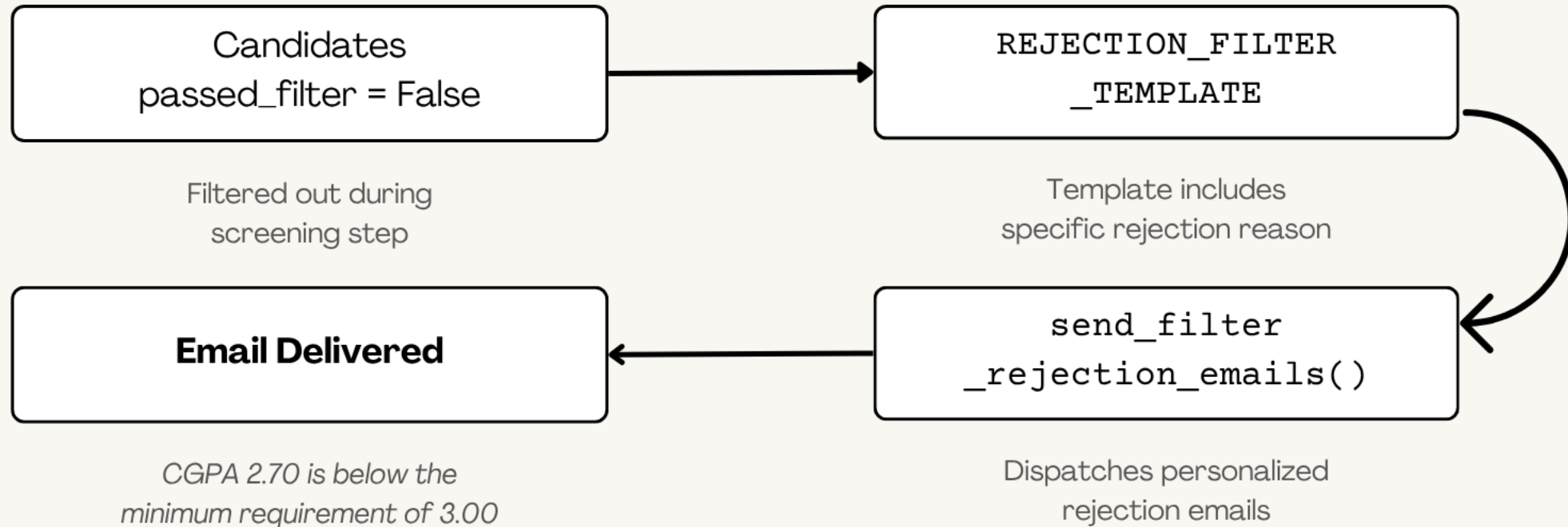
Node 05 — Action Dispatch

For the Filtered Out Candidates

`modules/email_dispatch.py`

smtplib

gsread



Node 06 — Background Automation

Full Pipeline Automation

`modules/scheduler_task.py`

smtpplib

gsread

The Challenge

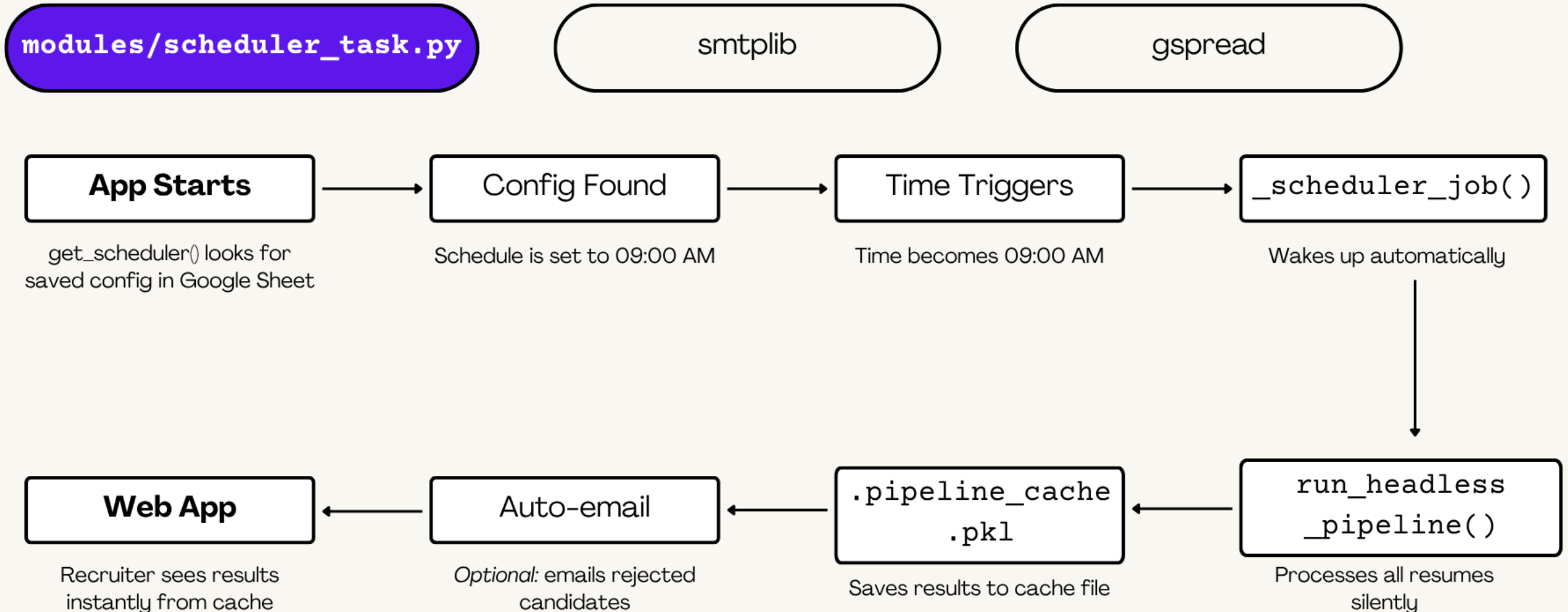
Manual execution for every new batch is inefficient

The Process

- Entire 5-step pipeline runs completely unattended
- Scheduled to wake up **automatically** (e.g., 2:00 AM)
- Fetches applications, ranks them, and exports results
- Recruiter wakes up to a fully processed, ready-to-review dashboard

Node 06 — Background Automation

Running the whole pipeline overnight, no clicks needed



Node 06 — Background Automation

Running the whole pipeline overnight, no clicks needed

`modules/pipeline_core.py`

`run_headless_sheet_pipeline()`

`fetch_candidates
_from_sheet()`

`ingestion.py`

`apply_knockout _filters()`

`ingestion.py`

`download_all _resumes()`

`ingestion.py`



`compute_summary _stats()`

`embedding.py`

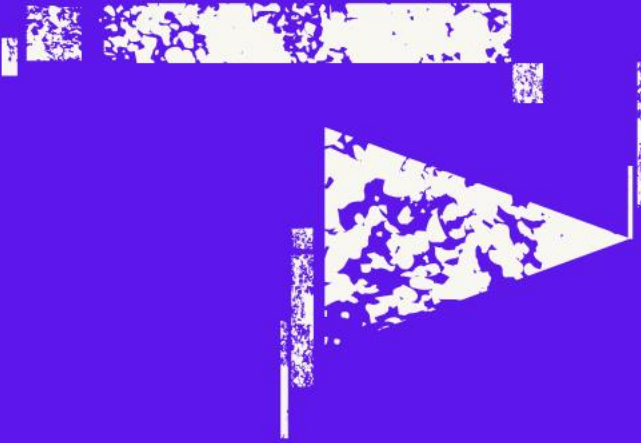
`rank_resumes _semantic()`

`embedding.py`

`extract_markdown
_from_pdf()`

`parser.py`

Returns → (DataFrame, stats, passed, filtered)



Thank You

Any questions?

